

# Infrastruktur & DevOps

## Table of Contents

|                                                                         |   |
|-------------------------------------------------------------------------|---|
| Hosting-Übersicht .....                                                 | 2 |
| API Deployment (Railway) .....                                          | 2 |
| Endpoints (Auth-frei) .....                                             | 3 |
| Production URLs (für <code>.env</code> -Dateien) .....                  | 3 |
| Frontends (Cloudflare Pages) .....                                      | 3 |
| Shared Types Publishing (GitHub Packages) .....                         | 4 |
| Secrets (Railway-Dashboard / Cloudflare-Secrets / GitHub Secrets) ..... | 4 |
| CI/CD-Hinweise .....                                                    | 4 |
| GitHub Packages Auth .....                                              | 4 |
| ESLint .....                                                            | 5 |
| Pre-Commit-Hook Version-Bump .....                                      | 5 |
| Workers-Learnings (legacy, vor Railway-Migration) .....                 | 5 |
| <code>process.exit()</code> vermeiden .....                             | 5 |
| Config lazy laden .....                                                 | 5 |
| Dynamischer <code>import()</code> — Response-Mismatch .....             | 6 |
| Testing-Strategie .....                                                 | 6 |
| Monitoring & Logging .....                                              | 6 |
| Repository-Topologie (8 Repos) .....                                    | 7 |
| Disaster Recovery .....                                                 | 7 |
| Security Best Practices .....                                           | 7 |
| Detail-Doku .....                                                       | 8 |
| Flammenreiter — Deployment & Infrastruktur .....                        | 8 |
| .1. Cloudflare Workers Deployment .....                                 | 8 |
| .2. Endpoints .....                                                     | 8 |
| .3. Production URLs (alle Repos) .....                                  | 8 |
| .4. Secrets (Cloudflare Worker Dashboard / wrangler secret put) .....   | 9 |
| .5. CI/CD Hinweise .....                                                | 9 |
| .6. Workers-spezifische Learnings .....                                 | 9 |



Live-Stand 2026-05-11. Diese Dokumentation beschreibt den **aktuellen** Deploy-Stack des Flammenreiter-Projekts. Frühere Versionen dieses Dokuments waren ein PLACEHOLDER — die folgenden Abschnitte spiegeln den Live-Stand wider.

## Hosting-Übersicht

| Komponente                | Platform                                                                        | Domain / URL                                                                                  |
|---------------------------|---------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| Datenbank                 | Supabase PostgreSQL (Pooler aws-1-eu-central-1)                                 | (interner Connection-String, siehe Secrets)                                                   |
| API (Hono)                | Railway (auto-deploy on push-to-main)                                           | <a href="https://api.flammenreiter.gmolike.me">https://api.flammenreiter.gmolike.me</a>       |
| Frontend Web (GM)         | Cloudflare Pages                                                                | <a href="https://gm.flammenreiter.gmolike.me">https://gm.flammenreiter.gmolike.me</a>         |
| Frontend Web (Player)     | Cloudflare Pages                                                                | <a href="https://player.flammenreiter.gmolike.me">https://player.flammenreiter.gmolike.me</a> |
| Frontend Web (Lore-Admin) | Cloudflare Pages                                                                | <a href="https://lore.flammenreiter.gmolike.me">https://lore.flammenreiter.gmolike.me</a>     |
| Shared Types (npm)        | GitHub Packages ( <a href="https://npm.pkg.github.com">npm.pkg.github.com</a> ) | @flammenreiter/api-types (v3.x)                                                               |
| Mobile (UE5)              | Manuell — App Store / Google Play (Future)                                      | n/a                                                                                           |
| AI / FLUX                 | fal.ai (Pay-as-you-go)                                                          | <a href="https://fal.run/fal-ai/flux-pro">https://fal.run/fal-ai/flux-pro</a>                 |



ue5/ Repo ist scaffolded aber **nicht** aktiv in Entwicklung (Workflow-Default §8).

## API Deployment (Railway)

Der **api/-**Service (Hono 4.6 + Prisma 6) wird automatisch deployed:

1. Push auf **api/main**
2. Railway erkennt Commit → Build (**pnpm install**, **pnpm build**)

3. Prisma-Migrationen werden bei Deploy nicht automatisch angewendet (manueller Schritt via `pnpm prisma migrate deploy`)
4. Health-Check auf `/health` (status + version, ohne Auth)
5. Custom-Domain `api.flammenreiter.gmolike.me` aktiv via CNAME



Push auf `api/main` triggert sofortigen Deploy. Vor Push immer Tests laufen lassen.

## Endpoints (Auth-frei)

| URL                                                                                                               | Beschreibung                        |
|-------------------------------------------------------------------------------------------------------------------|-------------------------------------|
| <a href="https://api.flammenreiter.gmolike.me/health">https://api.flammenreiter.gmolike.me/health</a>             | Health-Check (status + api-version) |
| <a href="https://api.flammenreiter.gmolike.me/version">https://api.flammenreiter.gmolike.me/version</a>           | api-version + shared-version        |
| <a href="https://api.flammenreiter.gmolike.me/swagger">https://api.flammenreiter.gmolike.me/swagger</a>           | Swagger UI (Dark Theme)             |
| <a href="https://api.flammenreiter.gmolike.me/openapi.json">https://api.flammenreiter.gmolike.me/openapi.json</a> | OpenAPI 3.1 Spec                    |

## Production URLs (für `.env`-Dateien)

```
API_URL=https://api.flammenreiter.gmolike.me
SESSION_GM_URL=https://gm.flammenreiter.gmolike.me
SESSION_PLAYER_URL=https://player.flammenreiter.gmolike.me
LORE_ADMIN_URL=https://lore.flammenreiter.gmolike.me
CORS_ORIGINS=https://gm.flammenreiter.gmolike.me,https://player.flammenreiter.gmolike.me,https://lore.flammenreiter.gmolike.me
```

## Frontends (Cloudflare Pages)

Alle drei React-Frontends (`session-gm`, `session-player`, `lore-admin`) werden auf Cloudflare Pages gehostet:

- *Build*: `pnpm build` → Vite-Output nach `dist/`
- *Deploy*: GitHub Action triggert auf Push-to-main → Cloudflare Pages Build
- *Custom-Domains*: pro Frontend ein CNAME (siehe Tabelle oben)
- *Auth*: Supabase Auth (JWKS-verified im api über JWT)

CI-Workflows der drei Frontends brauchen `NODE_AUTH_TOKEN` mit `read:packages`-Scope als GitHub-Action-Secret (für `@flammenreiter/api-types` aus GitHub Packages).

# Shared Types Publishing (GitHub Packages)

Schema-Änderungen fließen durch `shared/`:

1. Change Zod-Schema in `shared/src/schemas/...`
2. `pnpm changeset` → dokumentiert Bump (patch/minor/major) + Beschreibung
3. Merge auf `shared/main` + Tag `vX.Y.Z` → CI publishes auf GitHub Packages (`npm.pkg.github.com`)
4. Consumers (`api`, 3 Frontends) bumpen `@flammenreiter/api-types` in `package.json`
5. In `api`: `createRoute({ responses: { ... } })` an neue Schemas anpassen
6. In Frontends: `pnpm generate:api-types` regeneriert Consumer-Types

Aktuelle Version (2026-05-11): `v3.19.x` live.

## Secrets (Railway-Dashboard / Cloudflare-Secrets / GitHub Secrets)

Die folgenden Secrets sind kanonisch (`workspace-scripts/distribute-secrets.sh` propagiert sie an alle Sibling-Repos):

| Secret                                 | Quelle / Zweck                                                                                        |
|----------------------------------------|-------------------------------------------------------------------------------------------------------|
| <code>JWT_SECRET</code>                | Eigener Wert (api JWT-Signing)                                                                        |
| <code>DATABASE_URL</code>              | Supabase Connection String (pgbouncer / pooler)                                                       |
| <code>SUPABASE_URL</code>              | Supabase Project URL                                                                                  |
| <code>SUPABASE_ANON_KEY</code>         | Supabase Dashboard                                                                                    |
| <code>SUPABASE_SERVICE_ROLE_KEY</code> | Supabase Dashboard (nur api)                                                                          |
| <code>FAL_KEY</code>                   | fal.ai Access Key (für FLUX-Pipeline)                                                                 |
| <code>NODE_AUTH_TOKEN</code>           | GitHub PAT mit <code>read:packages</code> (für <code>pnpm install</code> von <code>api-types</code> ) |
| <code>SUPABASE_PW</code>               | Supabase-DB-Password (für direkte Migrations-Connections)                                             |

## CI/CD-Hinweise

### GitHub Packages Auth

CI-Workflows die `@flammenreiter/*` Pakete nutzen brauchen:

```
permissions:
```

```
contents: read
packages: read
```

Plus: Im Quell-Repo des Pakets unter **Package Settings** → **Manage Actions access** das konsumierende Repo mit "Read"-Rolle hinzufügen.

## ESLint

`typescript-eslint` muss als explizite devDependency installiert sein wenn `eslint.config.mjs` es importiert.

## Pre-Commit-Hook Version-Bump

Jedes Code-Repo (`api`, `shared`, `lore-admin`, `session-gm`, `session-player`) hat einen `pre-commit`-Hook (`scripts/version-bump.mjs`), der vor `lint-staged` läuft und das `version`-Feld in `package.json` SemVer-konform bumpet (siehe Workflow-Default §11).

## Workers-Learnings (legacy, vor Railway-Migration)

Diese Erkenntnisse stammen aus der Cloudflare-Workers-Phase des `api`-Repos (vor Umzug auf Railway). Sie bleiben relevant, falls einzelne Edge-Workflows künftig wieder auf Workers landen.

## `process.exit()` vermeiden

Crasht den Worker. Stattdessen `throw new Error()` verwenden.

## Config lazy laden

Environment Variables sind auf Workers erst nach dem ersten `fetch()` in `process.env` verfügbar. Config muss lazy geladen werden (z.B. via Proxy):

```
// FALSCH: Wird beim Import ausgeführt, env noch leer
const config = parseEnv(process.env)

// RICHTIG: Lazy via Proxy
export const config = new Proxy({}, {
  get(_, key) {
    if (!_parsed) _parsed = parseEnv(process.env)
    return _parsed[key]
  }
})
```

## Dynamischer `import()` — Response-Mismatch

Hono's Response ist nicht die native Workers-Response. Fix: `NativeResponse` vor dem Import capturen und Response wrappen:

```
const NativeResponse = Response

// ... nach app.fetch():
return new NativeResponse(res.body, {
  status: res.status,
  statusText: res.statusText,
  headers: res.headers,
})
```

## Testing-Strategie

| Test-Typ                 | Umfang                            | Tools             | Beispiele                                                    |
|--------------------------|-----------------------------------|-------------------|--------------------------------------------------------------|
| <i>Unit Tests</i>        | Hono Services,<br>Frontend Utils  | Vitest            | EffectProcessor,<br>DCCalculator                             |
| <i>Integration Tests</i> | api Routes + DB                   | Vitest + Test-DB  | RBAC, Session-Lifecycle                                      |
| <i>E2E Tests</i>         | User-Flows<br>(FRONTENDS PFLICHT) | Playwright        | Character-Picker, Live-<br>Session, Charakter-<br>Erstellung |
| <i>Snapshot Tests</i>    | UI-Komponenten                    | Vitest (selektiv) | Spider-Chart, Paper-<br>Doll, Character-Header               |



Workflow-Default §3: Jeder Bug-Fix und jedes neue Feature in einem Frontend braucht einen Playwright-E2E-Test. Unit-Tests beweisen Code-Korrektheit, nicht Feature-Korrektheit.

## Monitoring & Logging

- *Railway Logs*: api-stdout/stderr via Railway-Dashboard (**Logs** Tab)
- *Cloudflare Analytics*: Pro Frontend (Page-Views, Errors, TTFB)
- *Supabase Logs*: SQL-Queries, RLS-Violations via Supabase-Dashboard
- *fal.ai Dashboard*: FLUX-Pipeline Usage, Cost-Tracking

Custom-Alerting ist aktuell nicht aktiv — geplant für Sprint 2026-Q3.

# Repository-Topologie (8 Repos)

| Repo                    | Stack                                   | Deploy                                     |
|-------------------------|-----------------------------------------|--------------------------------------------|
| <b>workspace</b> (this) | Markdown only                           | not deployed                               |
| <b>shared</b>           | Zod 4 + tsup                            | GitHub Packages on <b>v*</b> tag           |
| <b>api</b>              | Hono 4.6 + Prisma 6 + PostgreSQL        | Railway auto-deploy on push-to-main        |
| <b>lore-admin</b>       | React 19 + Vite 6 + TanStack            | Cloudflare Pages                           |
| <b>session-gm</b>       | React 19 + Vite 6 + TanStack + D3       | Cloudflare Pages                           |
| <b>session-player</b>   | React 19 + Vite 5 + TanStack + Three.js | Cloudflare Pages                           |
| <b>docs</b> (this repo) | AsciiDoc + asciidoctor-pdf              | GitHub Artifacts + <b>build/pdf</b> branch |
| <b>ue5</b>              | Unreal Engine 5.4+ C++ + Blueprints     | Manual builds (dormant)                    |

## Disaster Recovery

| Szenario                   | Maßnahme                                                 | RTO      | RPO    |
|----------------------------|----------------------------------------------------------|----------|--------|
| <i>Database Corruption</i> | Supabase Automated Backups → Restore via Dashboard       | 30 min   | 1 hour |
| <i>Code Deployment Bug</i> | Railway Rollback via Deploy-History (one-click)          | 5 min    | 0 min  |
| <i>Frontend Bad Deploy</i> | Cloudflare Pages Rollback via Dashboard                  | 5 min    | 0 min  |
| <i>Security Breach</i>     | Supabase Service-Role-Key rotation + JWT_SECRET rotation | < 30 min | 0 min  |

## Security Best Practices

| Aspekt                    | Maßnahme                                                  |
|---------------------------|-----------------------------------------------------------|
| <i>Secrets Management</i> | GitHub Secrets + Railway Variables; keine Secrets im Code |

| Aspekt                  | Maßnahme                                            |
|-------------------------|-----------------------------------------------------|
| <i>RLS Enforcement</i>  | (geplant — aktuell läuft Auth via api-Layer-Checks) |
| <i>HTTPS Only</i>       | Alle Cloudflare/Railway-Endpoints HTTPS             |
| <i>CORS Policy</i>      | Whitelist explizit für gm/player/lore-domains       |
| <i>Input Validation</i> | Zod-Schema-Validation überall (api + Frontends)     |
| <i>SQL Injection</i>    | Prisma parameterisiert Queries automatisch          |

## Detail-Doku

- [deployment.adoc](#) — Workers-spezifische Notizen (legacy), kanonische Endpoints, Secrets-Liste

# Flammenreiter — Deployment & Infrastruktur

## .1. Cloudflare Workers Deployment

|                 |                                                                                           |
|-----------------|-------------------------------------------------------------------------------------------|
| Worker Name     | flammenreiter-api                                                                         |
| Custom Domain   | api.flammenreiter.gmlike.me                                                               |
| Workers.dev URL | flammenreiter-api.odya99.workers.dev                                                      |
| CF Account      | <a href="mailto:odya99@gmail.com">odya99@gmail.com</a> (b3856b7164cd3672fbfb2e13422ea1c0) |
| Login           | <code>npx wrangler login</code> (mit ody99-Account)                                       |
| Deploy          | <code>npx wrangler deploy</code>                                                          |

## .2. Endpoints

| URL                                                                                                             | Beschreibung                    |
|-----------------------------------------------------------------------------------------------------------------|---------------------------------|
| <a href="https://api.flammenreiter.gmlike.me/health">https://api.flammenreiter.gmlike.me/health</a>             | Health Check                    |
| <a href="https://api.flammenreiter.gmlike.me/swagger">https://api.flammenreiter.gmlike.me/swagger</a>           | Swagger UI (Dark Theme)         |
| <a href="https://api.flammenreiter.gmlike.me/openapi.json">https://api.flammenreiter.gmlike.me/openapi.json</a> | OpenAPI 3.1 Spec                |
| <a href="https://api.flammenreiter.gmlike.me/characters">https://api.flammenreiter.gmlike.me/characters</a>     | Characters CRUD (Auth required) |

## .3. Production URLs (alle Repos)

API\_URL=https://api.flammenreiter.gmlike.me



```
SESSION_GM_URL=https://gm.flammenreiter.gmolike.me
SESSION_PLAYER_URL=https://player.flammenreiter.gmolike.me
LORE_ADMIN_URL=https://lore.flammenreiter.gmolike.me
CORS_ORIGINS=https://gm.flammenreiter.gmolike.me,https://player.flammenreiter.gmolike.me,https://lore.flammenreiter.gmolike.me
```

## .4. Secrets (Cloudflare Worker Dashboard / wrangler secret put)

Diese Secrets sind im Worker gesetzt und muessen bei Aenderung aktualisiert werden:

| Secret                    | Quelle                     |
|---------------------------|----------------------------|
| JWT_SECRET                | Eigener Wert               |
| DATABASE_URL              | Supabase Connection String |
| SUPABASE_URL              | Supabase Project URL       |
| SUPABASE_ANON_KEY         | Supabase Dashboard         |
| SUPABASE_SERVICE_ROLE_KEY | Supabase Dashboard         |

## .5. CI/CD Hinweise

### .5.1. GitHub Packages Auth

CI-Workflows die `@flammenreiter/*` Pakete nutzen brauchen:

```
permissions:
  contents: read
  packages: read
```

Plus: Im Quell-Repo des Pakets unter **Package Settings** → **Manage Actions access** das konsumierende Repo mit "Read" Rolle hinzufuegen.

### .5.2. ESLint

`typescript-eslint` muss als explizite devDependency installiert sein wenn `eslint.config.mjs` es importiert.

## .6. Workers-spezifische Learnings

### .6.1. process.exit() vermeiden

Crasht den Worker. Stattdessen `throw new Error()` verwenden.

## .6.2. Config lazy laden

Environment Variables sind auf Workers erst nach dem ersten `fetch()` in `process.env` verfügbar. Config muss lazy geladen werden (z.B. via Proxy):

```
// FALSCH: Wird beim Import ausgeführt, env noch leer
const config = parseEnv(process.env)

// RICHTIG: Lazy via Proxy
export const config = new Proxy({}, {
  get(_, key) {
    if (!_parsed) _parsed = parseEnv(process.env)
    return _parsed[key]
  }
})
```

## .6.3. Dynamischer import() — Response Mismatch

Hono's Response ist nicht die native Workers Response. Fix: NativeResponse vor dem Import capturen und Response wrappen:

```
const NativeResponse = Response

// ... nach app.fetch():
return new NativeResponse(res.body, {
  status: res.status,
  statusText: res.statusText,
  headers: res.headers,
})
```